

QUESTIONS

Q1
Student Performance Analysis — List + Tuple
10 marks

Q2
Customer Name Analysis — String + List
10 marks

Q3
Product Inventory Analysis — List + Tuple
10 marks

Q4
Tour Analysis — String + List
10 marks

Q5
Employee Skill Analysis — Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple + List
10 marks

Q7
Password Security Analysis — String + List
10 marks

Q8
Code Analysis and Debugging — List + Tuple + String
10 marks

Q9
Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10
Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

Q1 Student Performance Analysis — List + Tuple

10 marks

A college stores the following student records:

```
students = [
    ('Arun', [78, 85, 92, 87]),
    ('Bala', [45, 67, 55, 49]),
    ('Charan', [90, 92, 88, 95]),
    ('Dhruv', [65, 72, 70, 68]),
    ('Esha', [88, 76, 91, 84])
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Dhruv : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Dhruv : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```
1 students = [
2     ('Arun', [78, 85, 92, 87]),
3     ('Bala', [45, 67, 55, 49]),
4     ('Charan', [90, 92, 88, 95]),
5     ('Dhruv', [65, 72, 70, 68]),
6     ('Esha', [88, 76, 91, 84])
7 ]
8
9 totals = {}
10 avg = {}
11 for name, marks in students:
12     t = sum(marks)
13     totals[name] = t
14     avg[name] = t/4
15     print(f"{name} -> total: {t}, avg: {t/4:.2f}")
16 print("Highest total:", max(totals, key=totals.get))
17 print("Lowest Avg:", min(avg, key=avg.get))
18 class_avg = sum(avg.values())/5
19 print("Above Class Avg:", [n for n, a in avg.items() if a > class_avg])
20 print("3+ marks > 80:", [n for n, m in students if len([s for s in m if s > 80]) >= 3])
21 print("Subject highest:", [max(s[i] for s, a in students) for i in range(4)])
22 print("Below 50:", [n for n, m in students if any(s<50 for s in m)])
23 sorted_s = sorted(students, key=lambda x: sum(x[1]), reverse=True)
24 print("Top 2:", [sorted_s[0][0], sorted_s[1][0]])
25 print("Immutable: tuple and names. mutable: marks list")
```

Input

5
Arun
78 85 92 87
Bala

Output

Arun -> total: 322, avg:
80.50
Bala -> total: 216, avg:
54.00
Charan -> total: 365, avg:
91.25

Run

Save

Q2
Customer Name Analysis —
String + List
10 marks

Q3
Product Inventory Analysis —
List + Tuple
10 marks

Q4
Tour Analysis — String + List
10 marks

Q5
Employee Skill Analysis —
Tuple + String
10 marks

Q6
Transaction Analysis — Tuple +
List
10 marks

Q7
Password Security Analysis —
String + List
10 marks

Q8
Code Analysis and Debugging —
List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

```
"Arun Kumar",  
"ARUN KUMAR",  
" arun kumar",  
"Bala Krishnan",  
"BALA KRISHNAN",  
"Charan",  
" charan",  
"DIVYA DEVI",  
"Divya Devi"  
]  
  
Analyze the customer data.  
  
Tasks  
  
1. Remove unnecessary leading and trailing spaces.  
2. Convert all names to lowercase.  
3. Identify duplicate customers.  
4. Count the number of unique customers.  
5. Find customers whose names contain more than one word.  
6. Find the longest customer name.  
7. Find customers whose names start with 'a' or 'd'.  
8. Count the number of characters in each normalized name.  
9. Sort the unique names alphabetically.  
10. Determine which customer name occurs most frequently.  
  
Expected Output:  
  
Normalized:  
[arun kumar, 'arun kumar', 'arun kumar',  
bala krishnan, 'bala krishnan',  
'charan', 'charan',  
'divya devi', 'divya devi']  
  
Unique Customers:  
[arun kumar, 'bala krishnan', 'charan', 'divya devi']  
  
Number of Unique Customers: 4  
  
Duplicate Customers:  
arun kumar  
bala krishnan  
charan  
divya devi  
  
Names with Multiple Words:  
arun kumar  
bala krishnan  
divya devi  
  
Longest Name:  
bala krishnan  
  
Starts with A or D:  
arun kumar  
divya devi
```

Your Code

```
1 names = [  
2     "Arun Kumar",  
3     "ARUN KUMAR",  
4     " arun kumar",  
5     "BALA KRISHNAN",  
6     "Charan",  
7     " charan",  
8     "DIVYA DEVI",  
9     "divya devi"  
10 ]  
11 normalized = [n.strip().lower() for n in names]  
12 print("Normalized:\n", normalized)  
13 unique = list(set(normalized))  
14 print("Unique Customers:\n", sorted(unique))  
15 print("Number of Unique Customers:")  
16 duplicates = [n for n in unique if normalized.count(n) > 1]  
17 print("Duplicate Customers:")  
18 for d in duplicates:  
19     print(d)  
20 multi = [n for n in unique if len(n.split()) > 1]  
21 print("Names with multiple words:")  
22 for m in multi:  
23     print(multi)  
24 print("Longest name:\n", max(unique, key=len))  
25 starts_ad = [n for n in unique if n[0] in ['a', 'd']]  
26 print("Names with A or D:")  
27 for s in starts_ad:  
28     print(s)  
29 print("Character Count:")  
30 for n in unique:  
31     print(n, ":", len(n))  
32 print("Sorted unique:", sorted(unique))  
33 print("Most frequent:", max(unique, key=lambda n: normalized.count(n))
```

Input

stdin input

Output

- 10 marks
- Q6 Transaction Analysis — Tuple + List
10 marks
- Q7 Password Security Analysis — String + List
10 marks
- Q8 Code Analysis and Debugging — List + Tuple + String
10 marks
- Q9 Word Frequency and Data Analysis — String + List + Tuple
10 marks
- Q10 Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 17500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 products = [
2     ("Laptop", 5, 55000),
3     ("mouse", 25, 700),
4     ("keyboard", 8, 1200),
5     ("monitor", 15, 12000),
6     ("printer", 4, 12000),
7     ("headset", 15, 2500)
8 ]
9 values = {}
10 print("Inventory values:")
11 for name, qty, price in products:
12     val = qty * price
13     values[name] = val
14     print(name + ": ", val)
15 total_val = 0
16 for v in values.values():
17     total_val = total_val + v
18 print("\nTotal inventory value:", total_val)
19 print("\nLow stock:")
20 for name, qty, price in products:
21     if qty < 10:
22         print(name)
23 print("\nPrice > 10000:")
24 for name, qty, price in products:
25     if price > 10000:
26         print(name)
27 most_exp = products[0]
28 for p in products:
29     if p[2] > most_exp[2]:
30         most_exp = p
31 print("\nMost expensive:\n", most_exp[0])
32
33 highest_name = ""
34 highest_val = 0
35 for name, val in values.items():
36     if val > highest_val:
37         highest_val = val
38         highest_name = name
39 print("\nHighest inventory value:\n", highest_val)
40
41 sorted_price = sorted(products, key=lambda x: x[2])
42 print("\nSorted by price:", [p[0] for p in sorted_price])
43
44 sorted_qty = sorted(products, key=lambda x: x[1])
45 print("\nSorted by quantity:", [p[0] for p in sorted_qty])
46
47 total_qty = 0
48 for name, qty, price in products:
49     total_qty = total_qty + qty
50 print("\nTotal Quantity:", total_qty)
51
52 print("\nInventory value > 50000:")
53 for name, val in values.items():
54     if val > 50000:
55         print(name)
```

Input

stdin input

Output

```
Inventory values:
Laptop: 275000
mouse: 17500
Keyboard: 9600
Monitor: 180000
Printer: 48000
```

CO4 AT2 CASE STUDY

← Back

QUESTIONS

- Q1 Student Performance Analysis — List + Tuple 10 marks
- Q2 Customer Name Analysis — String + List 10 marks
- Q3 Product Inventory Analysis — List + Tuple 10 marks
- Q4 Text Analysis — String + List 10 marks
- Q5 Employee Skill Analysis — Tuple + List + String 10 marks
- Q6 Transaction Analysis — Tuple + List 10 marks
- Q7 Password Security Analysis — String + List 10 marks
- Q8 Code Analysis and Debugging — List + Tuple + String 10 marks
- Q9 Word Frequency and Data Analysis — String + List + Tuple 10 marks
- Q10 Integrated Student Data Analysis — String + List + Tuple 10 marks

10/10 answered

Q4 Text Analysis — String + List

10 marks

A university receives the following feedback:

```
feedback = ""
```

Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better.

Analyze the text.

tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python
programming

is

learning
practice

Longest Word:

programming

Words containing 'p':

python
programming
powerful
practice

Your Code

```
1 feedback = "python programming is easy to learn.\npython programming is  
2 easy - feedback.lower()  
3 for ch in ".,!":  
4     text = text.replace(ch, "")  
5 words = text.split()  
6 print("total words:", len(words))  
7 unique = []  
8 for w in words:  
9     if w not in unique:  
10        unique.append(w)  
11 print("unique words:", len(unique))  
12 print("frequency of python:", words.count("python"))  
13 print("frequency of programming:", words.count("programming"))  
14 print("longest word:")  
15 for w in unique:  
16     if words.count(w) > 1:  
17         print(w)  
18 longest = unique[0]  
19 for w in unique:  
20     if len(w) > len(longest):  
21         longest = w  
22 print("longest word:", longest)  
23  
24 print("sorted unique words:", sorted(unique))  
25 print("words containing 'p':")  
26 for w in unique:  
27     if 'p' in w:  
28         print(w)  
29 percent = (len(unique) / len(words)) * 100  
30 print("percentage of unique words: {:.2f}%".format(percent))
```

Run

Save

Input

Output

```
total words: 18  
unique words: 12  
frequency of python: 3  
frequency of programming: 3  
repeated words:
```

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis —
String + List
10 marks**Q3**
Product Inventory Analysis —
List + Tuple
10 marks**Q4**
Test Analysis — String + List
10 marks**Q5**
Employee Skill Analysis —
Tuple + List + String
10 marks**Q6**
Transaction Analysis — Tuple +
List
10 marks**Q7**
Networked Security Analysis —
String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks**Q10**
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Employee Skill Analysis — Tuple + List + String

10 marks

An organization stores:

employees = [

('E101', 'Arun', ('Python', 'SQL', 'ML')),

('E102', 'Bala', ('Java', 'SQL', 'HTML')),

('E103', 'Charan', ('Python', 'ML', 'SQL')),

('E104', 'Divya', ('Python', 'Java', 'Cloud')),

('E105', 'Esha', ('Python', 'ML', 'Cloud'))

]

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```
1 employees = [
2     ('E101', 'Arun', ('python', 'SQL', 'ML')),
3     ('E102', 'Bala', ('java', 'SQL', 'HTML')),
4     ('E103', 'Charan', ('python', 'ML', 'SQL')),
5     ('E104', 'Divya', ('python', 'java', 'Cloud')),
6     ('E105', 'Esha', ('python', 'ML', 'Cloud'))
7 ]
8
9 def get_names(emp_list):
10     names = []
11     for _, name, _ in emp_list:
12         names.append(name)
13     return ", ".join(names)
14
15 p = [e for e in employees if "python" in e[2]]
16 print("Python:\n", get_names(p))
17
18 ps = [e for e in employees if "python" in e[2] and "ML" in e[2]]
19 print("\nPython + ML:\n", get_names(ps))
20
21 s = [e for e in employees if "SQL" in e[2] and "java" not in e[2]]
22 print("\nSQL but java:\n", get_names(s))
23
24 jc = [e for e in employees if "java" in e[2] or "Cloud" in e[2]]
25 print("\nJava or Cloud:\n", get_names(jc))
26
27 all_skill = []
28 for _, _, skills in employees:
29     all_skill = all_skill + skills
30
31 skill_count = {}
32 for sk in all_skill:
33     skill_count[sk] = skill_count.get(sk, 0) + 1
34
35 print("Number Common Skill:\n", max(skill_count, key=skill_count.get))
36
37 n3 = [e for e in employees if len(e[2]) >= 3]
38 print("\nEmployees with 3 skills:\n", get_names(n3))
39
40 np = [e for e in employees if "python" not in e[2]]
41 print("\nWithout python:\n", get_names(np))
42
43 sorted_emp = sorted(employees, key=lambda x: len(x[2]), reverse=True)
44 print("\nSorted by Skills:", [e[1] for e in sorted_emp])
45
46 print("\nSuitable for python + ML:\n", get_names(p))
```

Run

Save

Input

stdin input

Output

System error.

CO4 AT2 CASE STUDY

[← Back](#)

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis
— String + List
10 marks**Q3**
Product Inventory Analysis
— List + Tuple
10 marks**Q4**
Text Analysis
— String + List
10 marks**Q5**
Employee Skill Analysis
— Tuple + List + String
10 marks**Q6**
Transaction Analysis
— Tuple + List
10 marks**Q7**
Resource Security Analysis
— String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis
— String + List + Tuple
10 marks**Q10**
Integrated Student Data
Analysis
— String + List + Tuple
10 marks

10/10 answered

Q6 Transaction Analysis — Tuple + List

10 marks

A bank stores transactions:

```
transactions = [
    ("T101", "Deposit", 5000),
    ("T102", "Withdraw", 2000),
    ("T103", "Deposit", 8000),
    ("T104", "Withdraw", 1500),
    ("T105", "Deposit", 3000),
    ("T106", "Withdraw", 7000)
]
```

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than 14,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transaction = [
2     ("T101", "Deposit", 8000),
3     ("T102", "Withdraw", 2000),
4     ("T103", "Deposit", 8000),
5     ("T104", "Withdraw", 1500),
6     ("T105", "Deposit", 3000),
7     ("T106", "Withdraw", 7000)
8 ]
9
10 total_dep = 0
11 total_with = 0
12 dep_count = 0
13 with_count = 0
14 largest_dep = 0
15 largest_with = 0
16
17 for tid, ttype, amt in transaction:
18     if ttype == "Deposit":
19         total_dep = total_dep + amt
20         dep_count = dep_count + 1
21         if amt > largest_dep:
22             largest_dep = amt
23     else:
24         total_with = total_with + amt
25         with_count = with_count + 1
26         if amt > largest_with:
27             largest_with = amt
28
29 print("Total deposit:", total_dep)
30 print("Total withdrawal:", total_with)
31 print("Net balance:", total_dep - total_with)
32 print("Largest withdrawal:", largest_with)
33 print("Transaction > 4000:")
34
35 for tid, ttype, amt in transaction:
36     if amt > 4000:
37         print(tid)
38
39 print("Deposits:", dep_count)
40 print("Withdrawals:", with_count)
41 total_amt = total_dep + total_with
42 avg = total_amt / len(transaction)
43 print(f"Average transaction: {:.2f}", format(avg))
44 print(f"Sorted by Amount:")
45 sorted_transaction = sorted(transaction, key=lambda x: x[2])
46 for t in sorted_transaction:
```

Input

stdin input

Output

System error.

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis —
String + List
10 marks**Q3**
Product Inventory Analysis —
List + Tuple
10 marks**Q4**
Test Analysis — String + List
10 marks**Q5**
Employee Skill Analysis —
Tuple + List + String
10 marks**Q6**
Transaction Analysis — Tuple +
List
10 marks**Q7**
Password Security Analysis —
String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks**Q10**
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q7 Password Security Analysis — String + List

10 marks

A system receives:

```
passwords = [  
    'Python@123',  
    'python123',  
    'PYTHON@123',  
    'Py@45',  
    'Python123',  
    'Java@4567',  
    'Admin@2026'  
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

Expected Output:

Valid:

```
Python@123  
Java@4567  
Admin@2026
```

Invalid:

```
python123 → No uppercase, No special character  
PYTHON@123 → No lowercase  
Py@45 → Less than 8 characters  
Python123 → No special character
```

Valid Count: 3

Invalid Count: 4

Longest Password:

```
Python@123
```

Python-related passwords:

```
Python@123  
python123  
PYTHON@123  
Python123
```

Valid Percentage: 42.86%

Your Code

```
1 passwords = [  
2     'python@123',  
3     'python123',  
4     'PYTHON@123',  
5     'Py@45',  
6     'python123',  
7     'Java@4567',  
8     'Admin@2026'  
9 ]  
10 special_char = "!@#$%^&*()-_+=~[]{}|;:,.<@/?"  
11 valid_list = []  
12 invalid_list = []  
13 for pw in passwords:  
14     failed = []  
15     if len(pw) < 8:  
16         failed.append("Less than 8 characters")  
17     has_upper = False  
18     has_lower = False  
19     has_digit = False  
20     has_special = False  
21     for ch in pw:  
22         if ch.isupper(): has_upper = True  
23         if ch.islower(): has_lower = True  
24         if ch.isdigit(): has_digit = True  
25         if ch in special_char: has_special = True  
26     if not has_upper: failed.append("No uppercase")  
27     if not has_lower: failed.append("No lowercase")  
28     if not has_digit: failed.append("No digit")  
29     if not has_special: failed.append("No special")
```

Input

stdin input

Output

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis —
String + List
10 marks**Q3**
Product Inventory Analysis —
List + Tuple
10 marks**Q4**
Text Analysis — String + List
10 marks**Q5**
Employee Skill Analysis —
Tuple + List + String
10 marks**Q6**
Transaction Analysis — Tuple +
List
10 marks**Q7**
Password Security Analysis —
String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks**Q10**
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q8 Code Analysis and Debugging — List + Tuple + String

10 marks

Consider the following program:

```
records = [  
    ("Arun", "Python"),  
    ("ARUN", "Python"),  
    ("Bala", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]  
  
names = []  
  
for record in records:  
    name = record[0].strip().lower()  
  
    if name not in names:  
        names.append(name)
```

print(names)

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

```
['arun', 'bala', 'charan']
```

Your Code

```
1 records = [  
2     ("Arun", "python"),  
3     ("Arun", "python"),  
4     ("Bala", "java"),  
5     ("Bala", "java"),  
6     ("Charan", "python")  
7 ]  
8  
9 names = []  
10 for record in records:  
11     name = record[0].strip().lower()  
12     if name not in names:  
13         names.append(name)  
14 print(names)
```

▶ Run

📁 Save

Input

stdin input

Output

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis
— String + List
10 marks**Q3**
Product Inventory Analysis
— List + Tuple
10 marks**Q4**
Test Analysis
— String + List
10 marks**Q5**
Employee Skill Analysis
— Tuple + List + String
10 marks**Q6**
Transaction Analysis
— Tuple + List
10 marks**Q7**
Restaurant Security Analysis
— String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis
— String + List + Tuple
10 marks**Q10**
Integrated Student Data
Analysis
— String + List + Tuple
10 marks

10/10 answered

Word Frequency and Data Analysis — String + List + Tuple

10 marks

Consider:

```
text = "data science uses data analysis and data visualization"  
Analyze the string using Python.
```

Tasks:

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:
(word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

```
['data', 'science', 'uses', 'analysis', 'and', 'visualization']
```

Frequency:

data → 3

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
1 text = "data science uses data analysis and data visualization"  
2 words = text.split()  
3 print("Total words:", len(words))  
4 unique = []  
5 for w in words:  
6     if w not in unique:  
7         unique.append(w)  
8 print("Unique words:\n", unique)  
9 freq = {}  
10 for w in words:  
11     if w in freq:  
12         freq[w] = freq[w] + 1  
13     else:  
14         freq[w] = 1  
15 print("Unrepeated words:")  
16 for w in freq:  
17     if freq[w] > 1:  
18         print(w)  
19 print("Frequency:")  
20 for w in freq:  
21     print(w, ">>", freq[w])  
22 freq = sorted(freq, key=freq.get)  
23 print("Sorted frequency:\n", freq)  
24 top_list = []  
25 for w in freq:  
26     top_list.append((w, freq[w]))  
27 sorted_top = sorted(top_list, key=lambda x: x[1], reverse=True)  
28 print("Sorted by frequency:\n", sorted_top)  
29 longest = words[0]  
30 for w in words:  
31     if len(w) > len(longest):  
32         longest = w  
33 print("Longest starting with 'd':")  
34 for w in unique:  
35     if w[0] == 'd':  
36         print(w)  
37 repeated_count = 0  
38 for w in freq:  
39     if freq[w] > 1:  
40         repeated_count = repeated_count + freq[w]  
41 percent = (repeated_count / len(words)) * 100  
42 print("Percentage of repeated words: {:.2f}%".format(percent))
```

Input

stdin input

Output

arun kumar → 274

Lowest Total:

hala → 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

Top 2:

arun kumar → 91.53

arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 87.60

Subject 2 → 73.40

Subject 3 → 78.40

Your Code

```
1 student = [
2     ("A001", "arun kumar", [78, 85, 88]),
3     ("A002", "hala", [45, 67, 93]),
4     ("A003", "arun kumar", [88, 91, 95]),
5     ("A004", "Charan", [72, 76, 81]),
6     ("A005", "chitra", [55, 48, 62])
7 ]
8 data = []
9 for sid, name, marks in student:
10     data.append([sid.strip().upper(), name.strip().lower(), marks])
11 print("\nUnsorted names:")
12 for x in data: print(x[0], x[1], x[2])
13 totals, avgs = [], []
14 for sid, name, marks in data:
15     t = sum(marks)
16     a = t/3
17     totals[name] = t
18     avgs[name] = a
19 print("\ntotals:")
20 for n, t in totals.items(): print(n, "_", t)
21
22 print("\naverages:")
23 for n, a in avgs.items(): print(n, "_", "{:.2f}".format(a))
24
25 print("\nsorted at least One:")
26 for x in data:
27     if min(x[2]) < 50: print(x[1])
28
29 print("\nhighest total:\n", max(totals, key=totals.get), "_", max(totals.get()))
30 print("\nlowest total:\n", min(totals, key=totals.get), "_", min(totals.get()))
31
32 print("\naverage > 75:")
33 for n, a in avgs.items():
34     if a > 75: print(n)
35
36 print("\nduplicate name:")
37 names = [x[1] for x in data]
38 print([n for n in names if names.count(n)>1][0])
39
40 sorted_data = sorted(data, key=lambda x: sum(x[2]), reverse=True)
41 print("\ntop 2:")
42 for i in range(2): print(sorted_data[i][1], "{:.2f}".format(sum(sorted_data[i][2])))
43
44 print("\nSubject-wise highest:")
45 for i in range(3): print("Subject", i+1, "--", max(x[2][i] for x in data))
46
47 print("\nSubject-wise average:")
48 for i in range(3): print("Subject", i+1, "{:.2f}".format(sum(x[2][i] for x in data)/len(data)))
49
50 count = 0
51 for x in data:
52     if len([n for n in x[2] if n>80]) >= 2: count += 1
53 print("\nCount >80 in 2+ subjects:", count)
54 top = max(data, key=lambda x: x[2][0])
55 print("\nhighest in Subject 1:", top[1], "--", top[2][0])
56 print("\n16. list of tuple best because tuple keeps id, name, marks top")
```

Run

Save

Input

stdin input

Output